

Neural Network-Based European Option Pricing under Black-Scholes SDE Dynamics

Project group

May 7, 2026

Abstract

This project studies whether a feed-forward neural network can accurately approximate the Black-Scholes pricing function for European call options. The problem is addressed in a controlled setting: data are generated synthetically from the analytical Black-Scholes formula, while Monte Carlo simulation is used as a numerical benchmark. The neural network is trained in a supervised learning framework to learn the mapping from market and contract parameters to the theoretical option price. The goal is not to build a real trading system, but to evaluate the role of a neural network as a pricing surrogate in a mathematically well-defined financial setting.

Contents

Abstract	ii
1 Introduction	1
2 Theoretical Framework	2
2.1 Underlying Asset Dynamics	2
2.2 European Call Option	2
2.3 Black-Scholes Formula	2
2.4 Monte Carlo Pricing	2
3 Methodology	4
3.1 Synthetic Dataset	4
3.2 Neural Network	4
3.3 Training and Validation	4
3.4 Metrics	5
4 Implementation	6
4.1 Code Structure	6
4.2 Computational Efficiency	6
4.3 Reproducibility	6
5 Experiments	7
5.1 Preliminary Experiment	7
5.2 Planned Visualizations	7
6 Preliminary Results	8
6.1 Monte Carlo Comparison	10
7 Conclusions and Next Steps	11

1 Introduction

Derivative pricing is one of the classical problems in quantitative finance. For European options, the Black-Scholes model provides a closed-form price under precise assumptions on the dynamics of the underlying asset, the market, and volatility [1]. The availability of an analytical solution makes Black-Scholes an ideal controlled environment for evaluating the accuracy of numerical methods and machine learning models.

The research question of the project is:

Can a feed-forward neural network accurately approximate the Black-Scholes pricing function for European call options?

The project compares three approaches:

- the analytical Black-Scholes formula, used as the ground truth;
- Monte Carlo simulation, used as a numerical benchmark;
- a feed-forward neural network, used as a supervised learning model and pricing surrogate.

The main objective is not to replace Black-Scholes in a setting where a closed-form formula is available, but to verify whether a neural network can learn a known pricing function. This is a useful first step for understanding neural pricing methods in more complex settings, where analytical formulas are unavailable or computationally expensive.

2 Theoretical Framework

2.1 Underlying Asset Dynamics

In the Black-Scholes model, the price of the underlying asset follows a geometric Brownian motion under the risk-neutral measure. This formulation is standard in continuous-time financial market modeling [4]:

$$dS_t = rS_t dt + \sigma S_t dW_t, \quad (2.1)$$

where S_t is the asset price at time t , r is the risk-free rate, σ is the constant volatility, and W_t is a standard Brownian motion.

The solution of the SDE is:

$$S_T = S_0 \exp \left[\left(r - \frac{1}{2}\sigma^2 \right) T + \sigma\sqrt{T}Z \right], \quad Z \sim \mathcal{N}(0, 1). \quad (2.2)$$

2.2 European Call Option

A European call option with strike K and maturity T has payoff:

$$\max(S_T - K, 0). \quad (2.3)$$

Under risk-neutral pricing, its value is the discounted expected payoff:

$$C = e^{-rT} \mathbb{E}^{\mathbb{Q}} [\max(S_T - K, 0)]. \quad (2.4)$$

2.3 Black-Scholes Formula

The analytical price of a European call option is:

$$C_{BS} = S_0 N(d_1) - K e^{-rT} N(d_2), \quad (2.5)$$

where $N(\cdot)$ is the cumulative distribution function of a standard normal random variable and:

$$d_1 = \frac{\ln(S_0/K) + (r + \frac{1}{2}\sigma^2) T}{\sigma\sqrt{T}}, \quad (2.6)$$

$$d_2 = d_1 - \sigma\sqrt{T}. \quad (2.7)$$

In this project, C_{BS} is used both as the supervised learning target and as the reference value for evaluation.

2.4 Monte Carlo Pricing

Monte Carlo simulation approximates the risk-neutral expectation by generating many realizations of S_T :

$$\hat{C}_{MC} = e^{-rT} \frac{1}{M} \sum_{i=1}^M \max(S_T^{(i)} - K, 0). \quad (2.8)$$

For sufficiently large M , $\hat{C}_{MC}$ converges to C_{BS} . In this project, Monte Carlo is used as a numerical benchmark and as a comparison point for the neural network.

3 Methodology

3.1 Synthetic Dataset

The dataset is generated by randomly sampling option and market parameters. Each observation contains:

$$x = (S_0, K, T, r, \sigma), \quad (3.1)$$

and the target is:

$$y = C_{BS}(S_0, K, T, r, \sigma). \quad (3.2)$$

The initial parameter ranges are:

Variable	Description	Range
S_0	initial underlying price	[50, 150]
K	strike price	[50, 150]
T	maturity in years	[0.1, 2]
r	risk-free rate	[0, 0.05]
σ	volatility	[0.1, 0.6]

Table 3.1: Parameter ranges used to generate the synthetic dataset.

Using synthetic data makes the problem controlled and allows the neural network error to be evaluated directly against the exact target function.

3.2 Neural Network

The model is a feed-forward neural network for regression. This choice is motivated by the fact that sufficiently large feed-forward networks are universal function approximators under general assumptions [2, 3]. The initial architecture is:

$$5 \rightarrow 64 \rightarrow 64 \rightarrow 32 \rightarrow 1, \quad (3.3)$$

with ReLU activation functions in the hidden layers. The network approximates the function:

$$f_{\theta}(S_0, K, T, r, \sigma) \approx C_{BS}. \quad (3.4)$$

The optimized loss is the mean squared error:

$$\mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^n (f_{\theta}(x_i) - y_i)^2. \quad (3.5)$$

3.3 Training and Validation

The data are split into training, validation, and test sets. Inputs are standardized using **StandardScaler**; the target is also scaled during training to improve numerical stability and convergence. The network is trained with Adam, early stopping, and a fixed random seed for reproducibility.

3.4 Metrics

The main metrics are:

- Mean Absolute Error (MAE);
- Root Mean Squared Error (RMSE);
- coefficient of determination R^2 ;
- MAPE computed only on prices greater than 1, to avoid instability when the theoretical option price is close to zero.

4 Implementation

The project is implemented in Python with a modular structure. The main libraries are NumPy, pandas, SciPy, scikit-learn, matplotlib, and PyTorch.

4.1 Code Structure

Listing 4.1: Main project structure.

```
src/nn_option_pricing/  
  black_scholes.py    # analytical formula and payoff  
  monte_carlo.py     # efficient Monte Carlo simulation  
  dataset.py         # synthetic data generation and storage  
  model.py           # PyTorch feed-forward neural network  
  training.py        # training loop, scaling, early stopping  
  evaluation.py      # regression metrics  
  plots.py           # visualizations  
  pipeline.py        # end-to-end experiment  
scripts/  
  run_experiment.py  # command-line entry point
```

4.2 Computational Efficiency

The Black-Scholes formula and the dataset generation procedure are implemented in vectorized form with NumPy. The Monte Carlo method uses the exact terminal distribution of the geometric Brownian motion at time T , avoiding unnecessary time discretization. Moreover, Monte Carlo simulation is batched both over options and over simulated paths, which keeps memory usage under control.

The neural network training uses PyTorch **DataLoader**, mini-batch gradient descent, input standardization, and target scaling. These choices make the code suitable both for quick development runs and for larger experiments.

4.3 Reproducibility

To make the experiments reproducible, the project fixes the random seeds of NumPy, Python, and PyTorch. The artifacts produced by an experiment include:

- the generated synthetic dataset;
- the model checkpoint;
- the input scaler;
- the target scaler;
- metrics in JSON format;
- diagnostic plots.

5 Experiments

5.1 Preliminary Experiment

A first intermediate experiment was run to verify that the pipeline was correct and that the neural network was able to learn the pricing function. The command used was:

Listing 5.1: Command for a preliminary experiment.

```
python scripts/run_experiment.py \  
  --n-samples 10000 \  
  --max-epochs 40 \  
  --batch-size 1024 \  
  --mc-n-paths 5000 \  
  --mc-evaluation-samples 128
```

This run does not necessarily represent the final experiment of the project; rather, it is an initial correctness check of the implementation. Final experiments may use more observations, more training epochs, and a larger number of Monte Carlo paths.

5.2 Planned Visualizations

The main visualizations are:

- training loss and validation loss;
- comparison between the Black-Scholes price and the neural network prediction;
- distribution of prediction errors;
- absolute error as a function of moneyness S_0/K ;
- absolute error as a function of maturity T ;
- absolute error as a function of volatility σ .

6 Preliminary Results

In the first intermediate experiment, the neural network obtained the following metrics on the test set:

Metric	Value
MAE	0.6952
RMSE	0.9114
R^2	0.9985
MAPE, prices > 1	10.0137%

Table 6.1: Preliminary neural network metrics against Black-Scholes prices.

The high R^2 value suggests that the neural network can already approximate the Black-Scholes pricing function very accurately. The average absolute errors remain limited relative to the overall range of option prices. MAPE is more delicate to interpret, because the prices of far out-of-the-money options may be close to zero; for this reason, it is computed only on prices greater than 1.

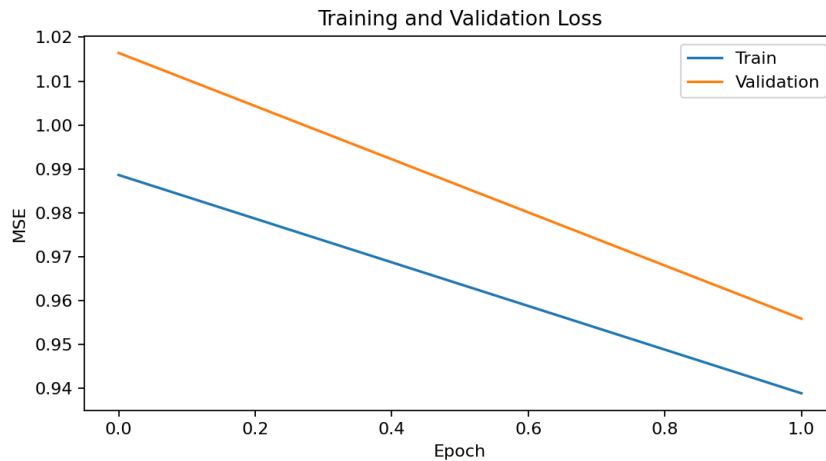


Figure 6.1: Training loss and validation loss.



Figure 6.2: Black-Scholes prices against neural network predictions.

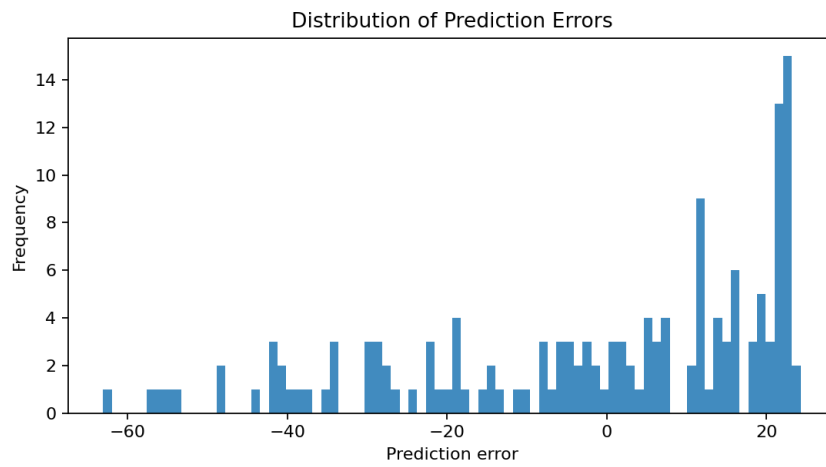


Figure 6.3: Distribution of prediction errors.

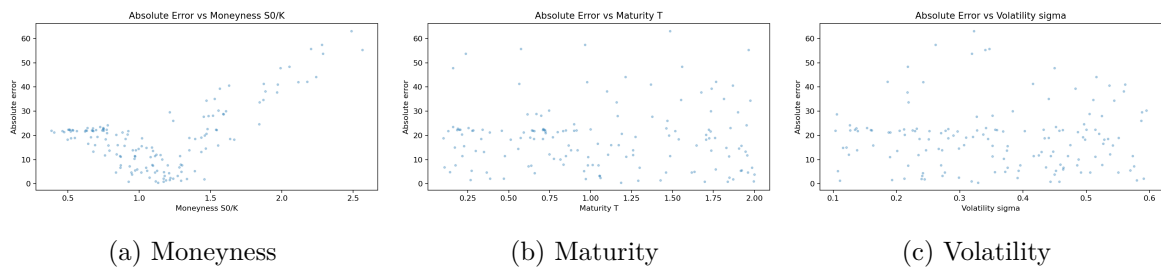


Figure 6.4: Absolute error against selected financial features.

6.1 Monte Carlo Comparison

On the subset used for the Monte Carlo benchmark, the numerical simulation obtained:

Metric	Value
MAE	0.3100
RMSE	0.5201
R^2	0.9996
MAPE, prices > 1	2.2139%

Table 6.2: Preliminary Monte Carlo metrics against Black-Scholes prices.

The comparison with Monte Carlo is useful because it highlights the difference between a simulation-based numerical method and a supervised model that, once trained, can produce predictions very quickly.

7 Conclusions and Next Steps

The first implementation confirms that a feed-forward neural network can approximate the Black-Scholes pricing function for European call options with high accuracy in a controlled synthetic environment. The project is therefore consistent with the initial research question and provides a solid basis for further experiments.

The next steps are:

- increase the size of the synthetic dataset;
- run longer training experiments and compare different architectures;
- study the error in specific regions of the parameter space, for instance deep in-the-money or out-of-the-money options;
- compare evaluation times across the analytical formula, Monte Carlo simulation, and the neural network;
- improve the theoretical discussion of the limitations of a pricing surrogate trained on synthetic data.

As future developments, one could consider stochastic volatility models, more complex options, or more advanced neural methods. These extensions are outside the scope of the first version of the project, but they represent natural directions for generalizing the approach.

Bibliography

- [1] Fischer Black and Myron Scholes. The pricing of options and corporate liabilities. *Journal of Political Economy*, 81(3):637–654, 1973.
- [2] George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2:303–314, 1989.
- [3] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989.
- [4] Steven E. Shreve. *Stochastic Calculus for Finance II: Continuous-Time Models*. Springer, 2004.